Intractability

Till now in computational complexity, we have discussed complexity classes **P**, **NP**, **co-NP**, and the notion of NP-hardness and NP-completeness via polynomial time mapping reductions. We also looked at the Cook-Levin Theorem which stated that CIRCUIT-SAT is NP-Complete. Using this as a starting point, here we will show many different problems to be NP-Complete, by reducing known NP-Complete problems to them.

How to show a problem to be NP-Complete?

Just to recap, we discussed the following strategy to show some language L to be NP-Complete:

1. Show that L is NP.
2. Choose a problem X which is known to be NP-Complete.
3. Prove that $X \leq_p^m L$. This involves describing a function f which maps from every instance of X to some instance of L, such that the YES instances of X are mapped to YES instances of L, and vice versa. Also, f should be computable in polynomial time.

3CNF-SAT is NP-Complete

We start our journey with the Boolean formula satisfiability problem. For now, we only know of one problem to be NP-Complete, CIRCUIT-SAT (from the Cook-Levin Theorem). We use it to show that 3CNF-SAT (and therefore SAT) is also NP-Complete.

A Boolean formula ϕ is said to be in **3CNF** form (conjunctive normal form with each clause with 3 literals), if it is a conjunction of clauses:

$$\phi = c_1 \wedge c_2 \wedge \ldots \wedge c_k$$

where each clause is a disjunction of 3 literals,

$$c_i = (l_{i1} \vee l_{i2} \vee l_{i3}).$$

A literal is a Boolean variable in its complemented or uncomplemented form: $l_j = x_i$, or $l_j = \overline{x_i}$.

The problem 3CNF-SAT determines, given a Boolean formula ϕ in 3CNF form over a set of n Boolean variables, whether there exists a satisfying assignment of variables.

For example, an instance of a 3CNF-SAT instance is:

$$\phi = (x_1 \vee \overline{x_2} \vee x_3) \wedge (\overline{x_1} \vee x_3 \vee \overline{x_4}) \wedge (x_2 \vee \overline{x_3} \vee \overline{x_4})$$

The above instance is satisfiable, an example of a satisfying assignment is $x_1 = 1, x_2 = 1, x_3 = 0, x_4 = 0$.

Notice that we are dealing with a very specific form of Boolean formulas. As we will show, determining satisfiability of a 3CNF-SAT instance is just as hard as determining satisfiability of any arbitrary Boolean circuit.[1]

3CNF-SAT is in NP

Clearly, we can have a polynomial time verifier for 3CNF-SAT with a satisfying assignment as a witness. The verifier substitutes the value for the variables with their values and evaluates each clause. If every clause evaluates to true (i.e., at least one literal in each clause is true), it returns true. Then, a formula ϕ is verifiable iff there exists a valid witness according to this verifier.

[1] This equivalence does not hold for many other forms of writing a Boolean formula; see Problem 1.

CIRCUIT-SAT $\leq_p^m$ 3CNF-SAT

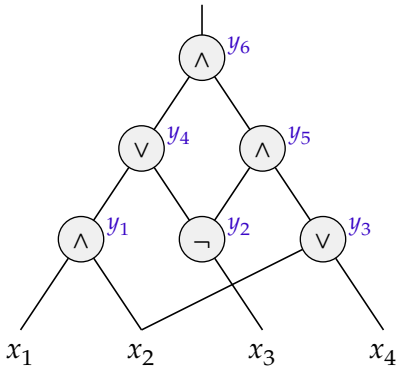
Now we need to show a procedure to convert any arbitrary Boolean circuit C in variables $\{x_i\}_{i=1}^n$ into a 3CNF formula ϕ, in such a way that

$$C \text{ is satisfiable if and only if } \phi \text{ is satisfiable.}$$

Furthermore, we should be able to perform this conversion in polynomial time, i.e. the time should be bounded by a polynomial in the size of the input circuit C.

First, WLOG let us assume that each internal $\vee$ and $\wedge$ node in C has a fan-in of 2. In general, there may be nodes with fan-in > 2, but they can be trivially converted into equivalent circuits with all nodes having two operands (e.g. $(a \vee b \vee c) \equiv ((a \vee b) \vee c)$). This conversion only increases the size of the circuit polynomially.

Introduce a new variable y_i for every internal node in the circuit. Then, we can write the Boolean formula ϕ as the AND of the root variable, and bi-implication clauses which indicate the operation of each internal node. For example, for a $\wedge$ node labeled y with two children labelled a and b, we have the clause $(y \leftrightarrow a \wedge b)$.



$$\begin{aligned}\phi = \quad & (y_1 \leftrightarrow x_1 \wedge x_2) \\ & \wedge (y_2 \leftrightarrow \neg x_3) \\ & \wedge (y_3 \leftrightarrow x_2 \vee x_4) \\ & \wedge (y_4 \leftrightarrow y_1 \vee y_2) \\ & \wedge (y_5 \leftrightarrow y_2 \wedge y_3) \\ & \wedge (y_6 \leftrightarrow y_4 \wedge y_5) \\ & \wedge y_6\end{aligned}$$

Figure 1: A Boolean circuit C and its corresponding Boolean formula ϕ.

ϕ is a conjunction of bi-implication clauses for each internal node, and a singleton clause for the root node. Clearly, the size of the formula is polynomial in the size of the circuit (in particular, the number of internal nodes in the circuit).

It is not in 3CNF form yet. To make it so, we can write each bi-implication clause by their equivalent CNF form by the truth table of the corresponding operation:

$$\begin{aligned}(y \leftrightarrow a \wedge b) &\equiv \left(\overline{a} \vee \overline{b} \vee y\right) \wedge (\overline{a} \vee b \vee \overline{y}) \wedge \left(a \vee \overline{b} \vee \overline{y}\right) \wedge (a \vee b \vee \overline{y}) \\ (y \leftrightarrow a \vee b) &\equiv \left(\overline{a} \vee \overline{b} \vee y\right) \wedge (\overline{a} \vee b \vee y) \wedge \left(a \vee \overline{b} \vee y\right) \wedge (a \vee b \vee \overline{y}) \\ (y \leftrightarrow \neg a) &\equiv (a \vee y) \wedge (\overline{a} \vee \overline{y})\end{aligned}$$

Note that these transformations replace each original clause by at most 4 clauses, thus only increases the size of the formula polynomially.

By applying these transformations, we can finally write ϕ as a CNF expression, where each clause has at most 3 literals. We need to do one final transformation so that every clause has exactly 3 distinct literals:

- If some clause C_i has 3 distinct literals, keep it as it is.

- If some clause C_i has 2 distinct literals, so $C_i = (l_1 \vee l_2)$, then add a new variable p, and write the clauses $(l_1 \vee l_2 \vee p) \wedge (l_1 \vee l_2 \vee \overline{p})$. Regardless of whether p takes the value 0 or 1, one of the clauses will evaluate $l_1 \vee l_2$, and the other to 1.
- If some clause C_i has 1 distinct literal, so $C_i = l$, add new variables q_1 and q_2, and write the clauses $(l \vee q_1 \vee q_2) \wedge (l \vee q_1 \vee \overline{q_2}) \wedge (l \vee \overline{q_1} \vee q_2) \wedge (l \vee \overline{q_1} \vee \overline{q_2})$. Again, regardless of what values q_1 and q_2 take, one of the new clauses will evaluate to l and all others to 1, hence the equivalence.

Again this step only increases the size of the formula polynomially. Thus we have obtained a 3CNF expression for ϕ.

Now, we need to show that C is satisfiable if and only if ϕ is satisfiable.

[$\Rightarrow$] Suppose C is satisfiable. Thus there exists an assignment of variables $\{x_i\}^n$ which makes the circuit evaluate to true. Then, one can simulate the circuit over this input and find the output of each internal node, labelled y_i. Keep the same assignment of x_is, and for y_is, assign them the value which the corresponding internal node outputs. (For other extra variables, assign them any value). This is a satisfying assignment for ϕ. If we think of the original bi-implication form of ϕ, then clearly the singleton root node clause is satisfied (since the root node of the circuit should be 1 as C is satisfiable). Also, every other internal node's clause must be satisfied, because they simply represent the operation that happens on that node during the circuit's simulation. Hence, ϕ is satisfiable.

[$\Leftarrow$] Suppose ϕ is satisfiable. Again, thinking of the original form, this means that each biimplication clause is satisfied, which essentially means that for any clause $y \leftrightarrow a \wedge b$, it is satisfied, so the assignment is such that $y = a \wedge b$. Similarly this is true for other nodes as well. Furthermore, the singleton clause for the root node is also satisfied, this means that the root node of the circuit outputs 1. Thus our formula ϕ is simply simulating the circuit C (the y_i values must exactly take the outputs of the corresponding nodes when the circuit is simulated) and hence we have a satisfying assignment for C, which is simply the values for $\{x_i\}$ in ϕ's satisfying assigment.

This proves that 3CNF-SAT is NP-Complete. ■

At this point, we know that SAT, and also in particular, 3CNF-SAT are NP-Complete problems. Now, let us discuss some other problems which are of more practical interest.

CLIQUE is NP-Complete

In an undirected graph $G = (V, E)$, a **clique** is a subset of vertices $S \subseteq V$, such that every two vertices in S are adjacent, i.e. $\forall u, v \in S, u \neq v, (u, v) \in E$. In other words, the induced subgraph of S in G is a complete graph. The **size of the clique** is the cardinality of the set S.

The clique problem is the optimisation problem of finding a clique of maximum size in a graph. As a decision problem, we formulate CLIQUE: Given a graph G and an integer k, determine whether G contains a clique of size k.

A naive algorithm for determining whether G has a clique of size k is to list all k-subsets of V, and check each one to see if it forms a clique. Since k in general can be $\Theta(n)$, this algorithm runs in superpolynomial time. We show that in fact, this problem is NP-Complete, implying that an efficient algorithm for this problem is unlikely to exist.

CLIQUE is in NP

For a given graph $G = (V, E)$, we use the subset S of vertices in a clique as a witness for G. We can check $|S| = k$, and check if S forms a clique in G by checking whether for each pair of vertices $u, v \in S$, the edge (u, v) is present in G.

We now show a reduction from 3-satisfiability to the clique problem, thus showing CLIQUE to be NP-hard. This seems unnatural, since both problems are of widely different types. One is about constraint satisfaction, while the other is a graph problem. We will need to show a mapping from any arbitrary instance of 3CNF-SAT into an instance of CLIQUE such that their membership in the language is equivalent. Doing so illustrates the general principle for designing reductions between problems $A \leq_p^m B$: we build "gadgets" out of components in B, which encode the information in the instance, and represent what is going on in problem A.

3CNF-SAT $\leq_p^m$ CLIQUE

Given an arbitrary 3CNF formula ϕ, we need to construct a graph G and a parameter k, such that

$$\phi \text{ is satisfiable if and only if } G \text{ has a clique of size } k.$$

Let $\phi = C_1 \wedge C_2 \wedge \ldots \wedge C_k$ be the Boolean formula (set k to be the number of clauses). Each clause C_i has three distinct literals, $C_i = (l_{i1} \vee l_{i2} \vee l_{i3})$.

We construct the graph G as follows: For every clause $C_i = (l_{i1} \vee l_{i2} \vee l_{i3})$, add three vertices corresponding to each of the three literals in C_i. Add an edge between two vertices corresponding to l_{ir} and l_{js} if:

- $i \neq j$, i.e. they do not belong to the same clause, and
- the literals are consistent, so l_{ir} and l_{js} are not negations of each other.

Clearly, this graph can be built in polynomial time. For example, if $\phi = (x_1 \vee \overline{x_2} \vee \overline{x_3}) \wedge (\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee x_3)$, then the corresponding graph is shown in Figure 2.

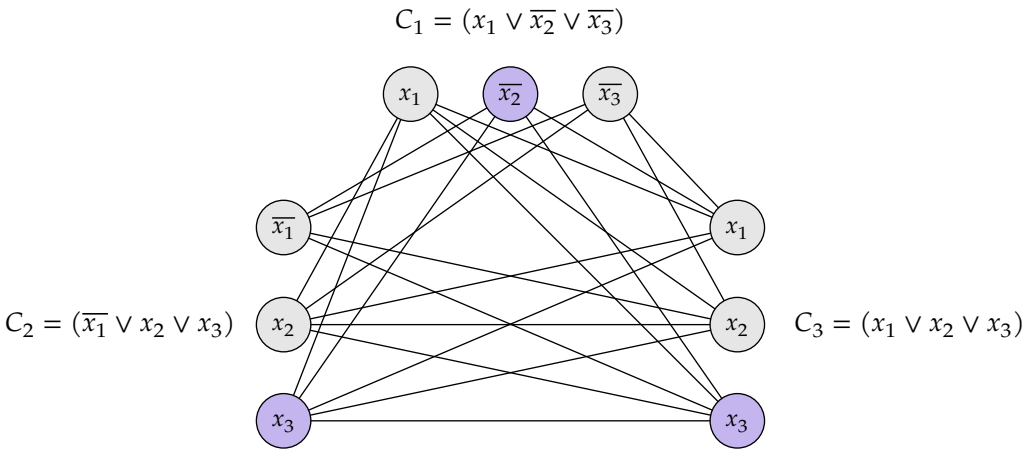


Figure 2: Graph G for $\phi = (x_1 \vee \overline{x_2} \vee \overline{x_3}) \wedge (\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee x_3)$ for the reduction 3CNF-SAT $\leq_p^m$ CLIQUE. The subset S (shaded purple) forms a clique in G.

We now need to show, that ϕ is satisfiable if and only if G has a clique of size k.

[$\Rightarrow$] Suppose ϕ is satisfiable. For some satisfying assignment, each clause is satisfied independently, thus each clause C_i contains at least one literal l_{ir} which is true. Pick one true literal from each clause. This gives a set S of k vertices in the graph. We claim that S is a clique: For any two vertices which correspond to l_{ir} and l_{js} in S, clearly $i \neq j$ since we picked one vertex from each

clause. Also, we know that these literals are true under our satisfying assignment. So it cannot be the case that they are negations of each other (both can't be true simultaneously). Thus, by our construction, there exists an edge between these two vertices in our graph G.

[$\Leftarrow$] Suppose G has a clique S of size k. Note that from a single triplet of vertices, no more than one vertex may be chosen in S, this is because vertices within the same triplet don't share an edge. Since the size of the clique is k, exactly one vertex (corresponding to literal l_{ir}) must have been chosen from each clause C_i. Create an assignment of variables such that each of these chosen literals becomes true under this assignment.

- Firstly, this is a valid assignment. This is because no two literals can be inconsistent (negations of each other) since they won't have an edge between them in G. So we can get a consistent assignment which satisfies all the chosen literals.
- Next, this is a satisfying assignment for ϕ. This is because in each clause, the chosen literal is true, so each clause evaluates to true and thus the entire formula.

This completes the proof that CLIQUE is NP-complete. ■

SUBSET-SUM is NP-Complete

Now we take a look at an arithmetic problem. In the **subset-sum** problem, we are given a list of positive integers $a_1, a_2, ..., a_n$, and a target integer $t > 0$. We wish to know whether there exists a subset of these numbers which add up to t. In other words, we want to find a subset $S \subseteq \{1, 2, ..., n\}$ such that $\sum_{i \in S} a_i = t$. (As usual, we assume all numbers are written in binary, so the input size is polynomial in n and $\log t$.)

This is where the encoding of numbers matters. A simple dynamic programming algorithm solves SUBSET-SUM in $O(nt)$ time, but this is not polynomial, since t can be exponential in the number of bits used to write it. We show that, unless **P** = **NP**, no algorithm can be polynomial in the *bit length* of the input.

SUBSET-SUM is in NP

For an instance $(a_1, ..., a_n, t)$, we use the index set S as a witness. The verifier computes $\sum_{i \in S} a_i$ and checks whether it equals t. This takes time polynomial in the bit length of the input, since we add at most n numbers, each with polynomially many bits.

3CNF-SAT $\leq_p^m$ SUBSET-SUM

Given an arbitrary 3CNF formula ϕ, we need to construct a list of numbers and a target t, such that

$$\phi \text{ is satisfiable if and only if some subset of the numbers sums to } t.$$

Let ϕ be a formula over variables $x_1, ..., x_n$ with clauses $C_1, ..., C_k$. The idea is to build numbers whose digits act as counters. A subset of the numbers will encode a truth assignment, and the digits will tell us whether every clause has been satisfied.

We will write the numbers in base 10 (for construction) and they will have $n + k$ digits. The first n digits are labelled by the variables $x_1, ..., x_n$, and the last k digits by the clauses $C_1, ..., C_k$. We create $2n + 2k$ numbers:

- For every variable x_i, two numbers y_i and z_i, corresponding to setting x_i true and false respectively.
- For every clause C_j, two *slack* numbers g_j and h_j, required for padding purposes.

	x_1	x_2	...	x_n	C_1	C_2	...	C_k
y_1	1	0	...	0	①		...	
z_1	1	0	...	0	0		...	
y_2	0	1	...	0	0		...	
z_2	0	1	...	0	①		...	
⋮	⋮	⋮	⋱	⋮	⋮	⋮	⋱	⋮
y_n	0	0	...	1	0		...	
z_n	0	0	...	1	①		...	
g_1					1	0	...	0
h_1					1	0	...	0
g_2					0	1	...	0
h_2			0		0	1	...	0
⋮					⋮	⋮	⋱	⋮
g_k					0	0	...	1
h_k					0	0	...	1
t	1	1	...	1	3	3	...	3

Figure 3: Constructing the list of numbers for the reduction 3CNF-SAT $\leq_p^m$ SUBSET-SUM. Row y_i stands for $x_i = 1$ and z_i for $x_i = 0$. In the first $2n$ numbers, each clause column has 3 ones, corresponding to its three literals. The two slack rows g_j, h_j both have a single 1 in column C_j.

We fill the digits as follows:

- For each variable x_i, the corresponding digit in the numbers y_i and z_i is set to 1, and 0 in all the other numbers.
- In the column of each clause C_j, ones are set corresponding to the y_i or z_i rows which correspond to the literals in C_j, y_i representing x_i and z_i representing $\overline{x_i}$. So in each column for a clause C_j, exactly 3 ones will be present in the y_i/z_i rows.
- We add the slack numbers, where the digits corresponding to each variable are 0, and we have 2 more ones in each clause column, in the corresponding slack numbers.

The target t has digit 1 in each of the n variable columns, and digit 3 in each of the k clause columns. This is illustrated in Figure 3. This construction takes polynomial time to create the $2(n + k)$ numbers having $n + k$ digits each.

Now we need to show that ϕ is satisfiable if and only if some subset of these numbers add up to t.

[⇒] Suppose ϕ is satisfiable. According to its satisfying assignment, for each i choose either y_i or z_i (choose y_i if $x_i = 1$, choose z_i otherwise.) Clearly, for the first n digits the sums match (1 in each column, since exactly one of y_i or z_i has been chosen). In the last k digits, in each column there must be at least one 1 that is chosen. This is because, since ϕ is satisfied, thus each clause C_j must be satisfied, so at least one of its literal must be true, and we have chosen the rows corresponding to all true literals. Then, the sum of the chosen numbers in the column for clause C_j can be 1, 2 or 3 (depending on how many of the clause's literals become true under the chosen assignment). If we are below 3, we can use the slack numbers g_j and h_j, choosing one or two of them to get that column's sum to exactly 3. Therefore, our chosen set of numbers exactly sums to the target t.

		x_1	x_2	x_3	C_1	C_2	C_3	C_4
y_1	=	1			1			1
z_1	=	1				1	1	
y_2	=		1					1
z_2	=		1		1	1	1	
y_3	=			1			1	1
z_3	=			1	1	1		
g_1	=				1			
h_1	=				1			
g_2	=					1		
h_2	=					1		
g_3	=						1	
h_3	=						1	
g_4	=							1
h_4	=							1
t	=	1	1	1	3	3	3	3

Figure 4: The instance for $\phi = (x_1 \vee \overline{x_2} \vee \overline{x_3}) \wedge (\overline{x_1} \vee \overline{x_2} \vee \overline{x_3}) \wedge (\overline{x_1} \vee \overline{x_2} \vee x_3) \wedge (x_1 \vee x_2 \vee x_3)$. The shaded rows are the subset corresponding to the satisfying assignment $x_1 = 1, x_2 = 0, x_3 = 0$. (Zeroes are not shown for clarity.)

[$\Leftarrow$] Suppose there exists a subset of numbers which add up to t. First, notice that there cannot be any carrying addition that takes place. This is because in each column the total sum of digits never exceeds 5. So every column adds up to the target's digit independently. The first n digits of the target are all 1, so this means that for each i, exactly one of y_i or z_i has been chosen in our subset. (If none are, or both are, then they don't sum to 1.) Construct the corresponding assignment based on which of y_i or z_i has been chosen: If y_i is chosen, set $x_i = 1$, otherwise, set $x_i = 0$. We claim this is a satisfying assignment.

Firstly, this assignment is clearly valid because for no i can both y_i and z_i be chosen so there are no inconsistencies. Next, consider the column corresponding to some clause C_j. Clearly, the chosen numbers in that column sum to 3. If we remove the slack rows (which can contribute at most 2 to the sum), and only consider the first $2n$ rows, then we see that the chosen numbers in this column must add up to at least 1. This means, that out of the 3 1s we placed for C_j corresponding to each of its literals, at least one of them has been chosen in the subset, and this means that literal is true in our assignment. If one literal is true in a clause, the clause becomes true. So, every clause C_j is true, hence ϕ is satisfiable.

This proves that SUBSET-SUM is NP-Complete. ■

VERTEX-COVER is NP-Complete

A **vertex-cover** of an undirected graph $G = (V, E)$ is a subset $S \subseteq V$ of vertices such that if $(u, v) \in E$ then $u \in S$ or $v \in S$ (or both). In other words, a vertex cover is a set of vertices that "covers" every edge in the graph. The **size** of a vertex-cover is the number of vertices in it.

The vertex-cover problem is to find a vertex-cover of minimum size in a given graph. As a decision problem, VERTEX-COVER asks to determine whether a graph G has a vertex cover of size k.

VERTEX-COVER is in NP

Given a graph $G = (V, E)$ and an integer k, we can use the vertex cover $S \subseteq V$ itself as the witness. The verifier affirms that $|S| = k$, and that for each edge $(u, v) \in E$, at least one of $\{u, v\}$ is in S. This clearly happens in polynomial time.

CLIQUE $\leq_p^m$ VERTEX-COVER

Given a graph G and an integer k, we need to produce a graph G' and an integer k' such that

$$G \text{ has a clique of size } k \text{ if and only if } G' \text{ has a vertex cover of size } k'.$$

Now this reduction is quite simple to describe since the problems are of the same kind. We first need to define the **complement** of a graph:

Given an undirected graph $G = (V, E)$, define the **complement** of G as $\overline{G} = \left(V, \overline{E}\right)$, where $\overline{E} = \{(u, v) \mid (u, v) \notin E, u \neq v\}$. In other words, $\overline{G}$ is the graph G containing the edges which are not in G.

The reduction algorithm takes an input instance $\langle G, k \rangle$ and computes the complement graph $\overline{G}$. The output of the reduction algorithm is simply $G' = \overline{G}$ and $k' = n - k$ (where $n = |V|$).

Now we need to show, that G has a clique of size k if and only if $\overline{G}$ has a vertex-cover of size $n - k$.

[$\Rightarrow$] Suppose G has a clique S of size k. Then, consider the subset $V \setminus S$. We claim that this is a vertex cover of $\overline{G}$, so it covers every edge in $\overline{E}$. Suppose for contradiction that it doesn't, i.e. there exists an edge $(u, v) \in \overline{E}$ such that $u \notin V \setminus S$ and $v \notin V \setminus S$. This means $u \in S$ and $v \in S$, but since S was a clique in G, we need $(u, v) \in E \Rightarrow (u, v) \notin \overline{E}$, a contradiction. So, $V \setminus S$ is indeed a vertex cover of $\overline{G}$.

[$\Leftarrow$] Suppose $\overline{G}$ has a vertex cover $V \setminus S$ of size $n - k$. Then, consider the subset S, we claim this is a clique in G. Suppose it is not, so there is some pair of vertices $u, v \in S$ such that $(u, v) \notin E \Rightarrow (u, v) \in \overline{E}$. We know that $V \setminus S$ is a vertex cover of $\overline{G}$ so at least one of u, v should be in $V \setminus S$, which is a contradiction. So S is a clique of G.

Hence, VERTEX-COVER is NP-Complete. ■

INDEPENDENT-SET is NP-Complete

An **independent set** in an undirected graph $G = (V, E)$ is a subset $S \subseteq V$ such that no two vertices in S are adjacent, i.e. $\forall u, v \in S, u \neq v, (u, v) \notin E$. As a decision problem, INDEPENDENT-SET asks: given a graph G and an integer k, does G contain an independent set of size k?

INDEPENDENT-SET is in NP

The witness is the set S. The verifier checks that $|S| = k$ and that no pair $u, v \in S$ has $(u, v) \in E$. This takes polynomial time.

CLIQUE $\leq_p^m$ INDEPENDENT-SET

The reduction is the same complement construction used for CLIQUE $\leq_p^m$ VERTEX-COVER. Given $\langle G, k \rangle$, output $G' = \overline{G}$ and $k' = k$. Computing the complement can be done in polynomial time.

We show that G has a clique of size k if and only if $\overline{G}$ has an independent set of size k. In fact, we will actually show for every $S \subseteq V$,

S is a clique in G if and only if S is an independent set in $\overline{G}$.

[$\Rightarrow$] Let S be a clique in G. Take any distinct $u, v \in S$. Since S is a clique, $(u, v) \in E$, so $(u, v) \notin \overline{E}$ by definition of the complement. Hence no two vertices of S are adjacent in $\overline{G}$, so S is an independent set in $\overline{G}$.

[$\Leftarrow$] Let S be an independent set in $\overline{G}$. Take any distinct $u, v \in S$. Then $(u, v) \notin \overline{E}$, which means $(u, v) \in E$. So every two vertices of S are adjacent in G, and S is a clique in G.

Since the sizes of S agree, G has a clique of size k if and only if $\overline{G}$ has an independent set of size k. This proves that INDEPENDENT-SET is NP-Complete. ■

Summary

Starting from CIRCUIT-SAT $\in$ **NPC** from the Cook-Levin theorem, we use reductions to prove unknown problems to be NP-Complete. Once we complete a reduction proof, the target problems get added to our pool of NP-Complete problems which we may use in the future to prove a new problem NP-Complete.

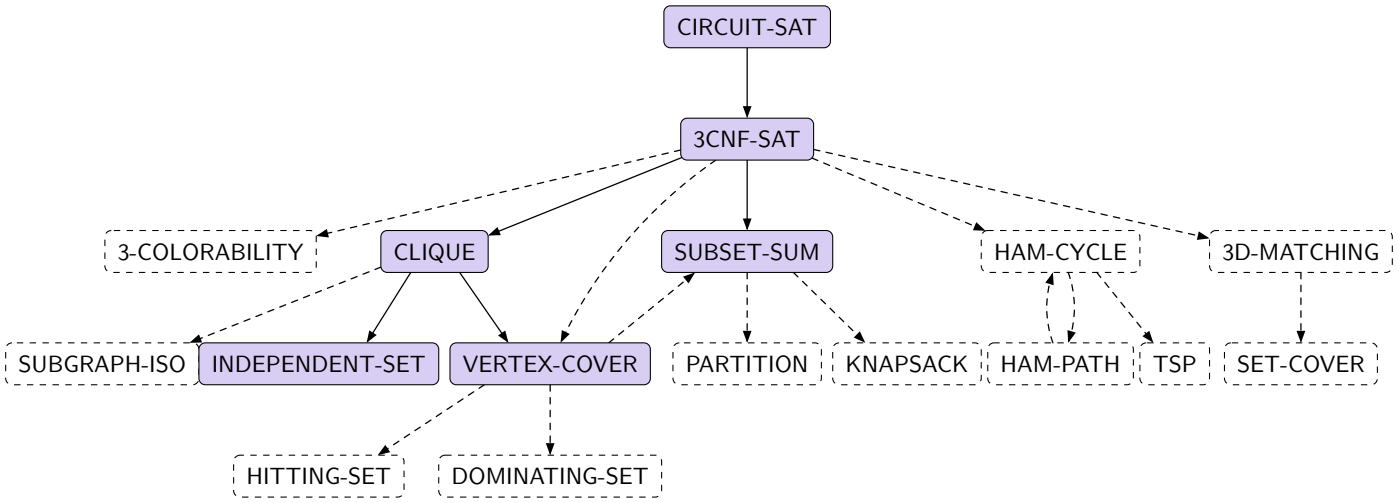


Figure 5: The zoo of NP-Complete problems. An arrow from A to B indicates a reduction $A \leq_p^m B$. Solid edges show reductions which we have shown, and dotted edges denote reductions asked in the problems.

All the problems shown in Figure 5 are actually NP-Complete. As of now, we have only proven the highlighted problems to be NP-Complete via the reductions drawn in solid lines. As we have seen, the common pattern to come up with these reductions is to build gadgets in the target problem that encode the choices and constraints of the source problem. The reductions shown in dotted lines are asked in the problems.

Problems and Solution Sketches

For standard reductions, refer to the course textbooks CLRS 3e (link) Chapter 34 and KT (link) Chapter 8.

1. Design a polynomial-time algorithm for each of the following problems: 2SAT, Eulerian Cycle, 2-Colorability of Graphs, DNF Satisfiability, and Minimum Edge Cover.

2SAT: Given a Boolean formula ϕ in 2CNF form (conjunction of clauses with exactly two literals), determine whether a satisfying assignment exists.

Suppose ϕ is over n variables $x_1, x_2, ..., x_n$ and has k clauses: $C_1 \wedge C_2 \wedge ... \wedge C_k$.

1. First, break each clause into implication clauses as follows:

$$(a \vee b) \quad \equiv \quad (\overline{a} \to b) \wedge \left(\overline{b} \to a\right)$$

2. Draw the **implication graph**: Draw two vertices corresponding to each variable x (one for its uncomplemented literal x and one for its complemented literal $\overline{x}$). Draw a directed edge from l_1 to l_2 if there is an implication clause of the form $(l_1 \to l_2)$.
3. Condense the implication graph into its strongly connected components (by using a polynomial time algorithm like Tarjan or Kosaraju).
4. For any variable x_i, if the literals x_i and $\overline{x_i}$ lie in the same component, then ϕ is NOT satisfiable. Otherwise, ϕ is satisfiable.

The algorithm runs in polynomial time.

We now show that ϕ is satisfiable if and only if no variable has both its literals in the same SCC.

First, we have the following observation:

- Under any satisfying assignment, if there is an edge $u \to v$ and u is true, then v is true. (Otherwise $\overline{u} \vee v$ would be false, but this is exactly the clause the edge came from.) By induction along a path, if there is a path from p to q and p is true, then q must be true.

[$\Rightarrow$] Suppose ϕ is satisfiable, with satisfying assignment α, and suppose for contradiction that some x has x and $\overline{x}$ in the same SCC. Then there are paths $x \rightsquigarrow \overline{x}$ and $\overline{x} \rightsquigarrow x$. If α sets x true, then by the observation, $\overline{x}$ is true, which is impossible. If α sets x false, then $\overline{x}$ is true, and so x is true by the second path, again impossible. This is a contradiction, so no such x exists.

[$\Leftarrow$] Suppose that for every variable, x_i and $\overline{x_i}$ lie in different SCCs. Let $\mathrm{SCC}(l)$ denote the strongly connected component containing literal l. Topologically sort the SCCs in the condensation graph. In the ordering, for each variable x, if $\mathrm{SCC}(x)$ occurs before $\mathrm{SCC}(\overline{x})$, assign $x = 0, \overline{x} = 1$. Otherwise, assign $x = 1, \overline{x} = 0$. We claim that this is a satisfying assignment.

Consider any clause $(a \vee b)$, and suppose for contradiction that it is false, so a and b are both false. Then $\overline{a}$ and $\overline{b}$ are true. By our construction, $\mathrm{SCC}(a)$ occurs before $\mathrm{SCC}(\overline{a})$, and $\mathrm{SCC}(b)$ occurs before $\mathrm{SCC}\left(\overline{b}\right)$. The clause gives the edges $\overline{a} \to b$ and $\overline{b} \to a$, so in the topological order

$$\mathrm{SCC}(\overline{a}) \leq \mathrm{SCC}(b) \quad \text{and} \quad \mathrm{SCC}\left(\overline{b}\right) \leq \mathrm{SCC}(a).$$

Chaining this with the other inequalities gives us

$$\mathrm{SCC}(a) < \mathrm{SCC}(\overline{a}) \leq \mathrm{SCC}(b) < \mathrm{SCC}\left(\overline{b}\right) \leq \mathrm{SCC}(a),$$

so $\mathrm{SCC}(a) < \mathrm{SCC}(a)$, a contradiction. Hence every clause is satisfied.

Thus we have shown a polynomial time algorithm for 2SAT. ■

Eulerian Cycle: Given an undirected graph $G = (V, E)$, determine whether there exists an Eulerian Cycle: a path which starts and ends on the same vertex, and visits every edge exactly once.

(For this discussion, we assume a 'path' and a 'cycle' can have repeated vertices, but not repeated edges.)

Algorithm:
1. If G is disconnected and has multiple connected components (ignoring isolated vertices), then G does not have an Eulerian cycle.
2. Find the degree of every vertex. If the degree of every vertex is even, then G has an Eulerian cycle. Otherwise, it does not.

Clearly this runs in $O(m + n)$ i.e. polynomial time.

We show that a connected graph G has an Eulerian cycle if and only if every vertex has even degree.

[$\Rightarrow$] Suppose G has an Eulerian cycle C. For any vertex v, all its incident edges appear once in C. Each time C passes through v, it must enter through one edge and exit through another. Thus, it is necessary that v has an even number of incident edges, so degree of v must be even.

[$\Leftarrow$] (Rough sketch) Show the other side of the proof, i.e. if every vertex has even degree, then G has an Eulerian cycle. This can be done by first showing:
- Lemma: In a graph G, if every vertex has even degree, then any (non isolated) vertex v lies on some cycle. (One can construct such a cycle by repeatedly taking some edge from the current vertex, at some point we reach the starting vertex again.)

Now, to prove the original statement, we make a contradiction argument. Suppose C is the largest cycle in G (largest by number of edges in the cycle). If C uses all edges in G, it is an Eulerian cycle and we are done. Otherwise, we consider the subgraph $G \setminus C$. In this subgraph, all vertices will have even degree (why?). Also, there is some vertex v which was a part of C and still has some edge left (why?). Applying the lemma on this vertex in $G \setminus C$, we obtain a new cycle C'. Since both cycles C and C' in G go through v, we could 'merge' the two cycles into one bigger cycle (how?), thus contradicting maximality of C. So C must be a Eulerian cycle.

Thus Eulerian cycle can be determined in polynomial time. ■

2-COLOR: Given an undirected graph $G = (V, E)$, determine whether it is 2-colorable.

A graph $G = (V, E)$ is said to be **2-colorable** if there exists a coloring $c : V \to \{0, 1\}$, such that for any edge $(u, v) \in E$, the two vertices have a different color, i.e. $c(u) \neq c(v)$.

Algorithm: Perform DFS on G. Arbitrarily color the first vertex (of each new component). For each edge (u, v) being explored,
- If v hasn't been visited yet, color it the opposite color of u.
- If v has been explored, and $c(u) = c(v)$, report G is NOT 2-colorable. Continue otherwise.

If we successfully complete the DFS then report G is 2-colorable.

The time complexity of this algorithm is linear in the size of the graph, so it runs in polynomial time.

First, we **claim**: If our algorithm reports that G is not 2-colorable, then G has a cycle of odd length. Say (u, v) is the edge where our algorithm gets stuck, so $c(u) = c(v)$. Consider the unique path from u to v in the DFS tree. By our coloring strategy, this path must be alternately colored. Since both u and v are colored same, the path should have an even number of edges. Then consider the cycle comprising of this path and the edge (u, v), this is an odd cycle.

We now show that G is 2-colorable if and only if the algorithm reports so.

[⇒] Let G be 2-colorable. Suppose our algorithm claims otherwise. Then, G has an odd cycle, say it is $\langle v_1, v_2, ..., v_{2k+1}, v_1 \rangle$. Since the graph is 2-colorable, G can be colored with 2 colors such that all adjacent pairs of vertices are differently colored.

WLOG, let $c(v_1) = 0$. Then, we have,

$$c(v_1) = 0 \Rightarrow c(v_2) = 1 \Rightarrow c(v_3) = 0 \Rightarrow ... \Rightarrow c(v_{2k}) = 1 \Rightarrow c(v_{2k+1}) = 0$$

But, $c(v_{2k+1}) = 0$ and $c(v_1) = 0$ while $(v_{2k+1}, v_1) \in E$, which is a contradiction. Thus G cannot contain a cycle of odd length. So our algorithm must report correctly, that G is 2-colorable.

[⇐] Suppose our algorithm reports that G is 2-colorable. This means that it managed to complete the DFS and 2-color the entire graph. Since during the DFS we iterate over all edges, and successful exit implies endpoints of all edges are differently colored, this means the algorithm has found a valid 2-coloring of G, thus G must be 2-colorable.

Therefore, 2-colorability is polynomial-time solvable. ■

DNF Satisfiability: Given a Boolean formula ϕ in DNF (disjunctive normal form), i.e. an OR of one or more clauses where each clause is an AND of one or more literals; determine whether ϕ is satisfiable.

Algorithm: If there exists some clause $C_i = l_1 \wedge l_2 \wedge ... \wedge l_m$ where no two literals are complements of each other, then ϕ is satisfiable. Otherwise, it is not.

We show ϕ is satisfiable if and only if there is some clause where no complementary pair of literals occur.

[⇒] Suppose ϕ is satisfiable, then under a satisfying assignment, at least one clause C_i must be satisfied. For this to happen, all literals in C_i must become true. Then, C_i cannot contain any complementary pair of literals, since they both can't be true at the same time.

[⇐] Suppose C_i is a clause where there is no complementary pair of literals, so all literals are 'consistent'. Then, assign each of these literals to 1. (Any variable that does not appear can be arbitrarily assigned). This assignment is well defined because no complementary pair of literals appear in C_i. This is a satisfying assignment because this makes all literals in C_i to be true, so C_i is satisfied, thus ϕ is satisfied.

Thus, DNF Satisfiability can be determined in polynomial time. ■

Minimum Edge Cover: Given an undirected graph G, find a minimum edge cover of G.

In an undirected graph $G = (V, E)$, an **edge cover** is a set of edges $S \subseteq E$ such that every vertex $v \in V$ is an endpoint of some edge $e \in S$.

Algorithm:
1. If G has any isolated vertices, no edge cover exists.
2. Find a maximum matching M in G (Edmonds' Blossom algorithm).
3. For each unmatched vertex v, pick one of its incident edges. Call this set U. Since M matches $2|M|$ vertices, $n - 2|M|$ vertices are unmatched. So $|U| = n - 2|M|$.
4. Output $S = M \cup U$ as the minimum edge cover. Its size is $|M| + (n - 2\,|M|) = n - |M|$.

(This is Gallai's theorem: If $\alpha'(G)$ is the size of a maximum matching in G, and $\beta'(G)$ is the size of a minimum edge cover, then we have $\beta'(G) = n - \alpha'(G)$.)

We now show that S is indeed a minimum edge cover.

1. **Every vertex is covered.** For any vertex v, if v is matched by M, then it is covered by an edge in M, so it is clearly covered. If v is unmatched in M, then we have added an edge incident on v in U, so it is also covered.
2. **Minimality of S.** Suppose we have an edge cover S' such that $|S'| < |S| = n - |M|$. Consider the graph with vertices of G and only edges in S'. Suppose this graph has k connected components. (There will be no isolated vertices in this graph, by virtue of S' being an edge cover.) Then, in each connected component of size c, the number of edges in it will be at least $c - 1$. Adding this across all the components, we see that $|S'| \geq n - k$, so $n - k < n - |M| \Rightarrow k > |M|$.* Now, we can create a set M' by choosing one edge in S' for each of the k connected components. M' is a matching in G, since clearly no two edges can be incident on the same vertex (we have chosen one edge from each connected component). But, $|M'| = k > |M|$, contradicting the maximality of M.

* (What this means is that originally S had $|M|$ connected components, now when we have a smaller edge cover S', it partitions G into more ($k > |M|$) connected components.)

Thus we have shown a polynomial time algorithm to find the minimum edge cover in a graph G. ■

2. Give an example of a problem which is NP-hard but unlikely to be NP-complete.

The Halting Problem is NP-hard, but not in NP (in fact, it is *undecidable*).

Halting Problem: Given a machine M and an input x, determine whether M halts on x.

Proof of NP-hardness: We can show SAT $\leq_p^m$ HP. This proves HP is at least as hard as SAT, so it is at least as hard as every NP problem, so it is NP-hard.

Given an arbitrary Boolean formula ϕ, we can construct a machine M which halts, if and only if ϕ is satisfiable.

Machine M works as follows:
1. It tries out every possible assignment of variables to check whether it satisfies ϕ.
2. If some satisfying assignment is found, the machine halts.
3. If all assignments are exhausted and no satisfying assignment exists, the machine goes into an infinite loop.

Construction of this machine can be done in polynomial time in the size of the instance ϕ.

Thus, HP is NP-hard.

3. Show that CNF-SAT many-to-one reduces to 3SAT.

CNF-SAT $\leq_p^m$ 3SAT: link

4. Show that 3SAT many-to-one reduces to Vertex Cover.

3SAT $\leq_p^m$ VERTEX-COVER

Given an arbitrary 3CNF formula ϕ, we need to construct a graph G and a parameter k, such that

$$\phi \text{ is satisfiable if and only if } G \text{ has a vertex cover of size } k.$$

Let $\phi = C_1 \wedge C_2 \wedge \ldots \wedge C_l$ contain n variables $x_1, \ldots, x_n$ and l clauses, where each clause $C_j = \left(l_{j1} \vee l_{j2} \vee l_{j3}\right)$ has three distinct literals. Set $k = n + 2l$. We construct a graph G with $2n + 3l$ vertices.

- **Variable gadget.** For every variable x_i, add two vertices labelled x_i and $\overline{x_i}$, joined by an edge.
- **Clause gadget.** For every clause C_j, add a triangle with three vertices labelled by the literals l_{j1}, l_{j2}, l_{j3}.

- Add **connection edges**: Join each vertex of a clause gadget to the vertex of the variable gadget carrying the same literal. For example, the clause vertex labelled $\overline{x_2}$ is joined to the vertex $\overline{x_2}$ of the gadget for x_2.

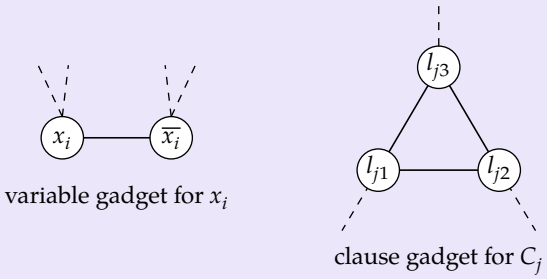


Figure 6: The gadgets for the reduction 3SAT $\leq_p^m$ VERTEX-COVER.

Clearly, this graph can be built in polynomial time.

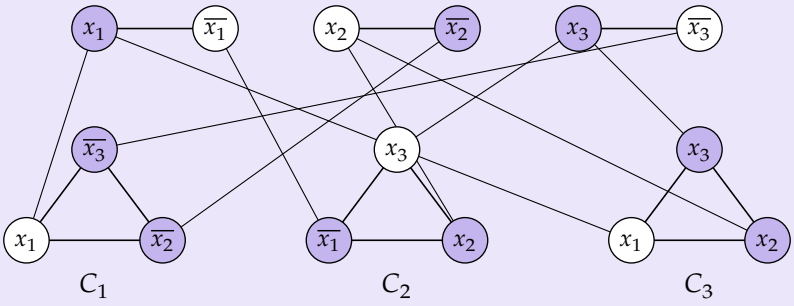


Figure 7: Graph G for $\phi = (x_1 \vee \overline{x_2} \vee \overline{x_3}) \wedge (\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee x_3)$ for the reduction 3SAT $\leq_p^m$ VERTEX-COVER.

We now need to show that ϕ is satisfiable if and only if G has a vertex cover of size $k = n + 2l$.

[$\Rightarrow$] Suppose ϕ is satisfiable, and fix a satisfying assignment. Build a set S as follows:
- From each variable gadget, pick the vertex whose literal is true. This gives n vertices.
- Each clause C_j contains at least one true literal. Pick one such triangle vertex and call it the *left-out* vertex. Pick the other two vertices of the triangle in S. This gives $2l$ vertices.

So $|S| = n + 2l = k$. We claim that S is a vertex cover, so every edge needs to be covered.
- **Variable edge** $(x_i, \overline{x_i})$**:** exactly one of the two literals is in S, so this edge is covered.
- **Clause edges:** two of the three vertices of every triangle are in S, so every triangle edge is covered.
- **Connection edges:** if the clause endpoint is in S, the edge is covered. Otherwise the clause endpoint is the left-out vertex, whose literal is true by our choice. Then its other endpoint, the variable-gadget vertex of this literal, is in S, so this edge is covered.

[$\Leftarrow$] Suppose G has a vertex cover S of size $n + 2l$. The n variable gadgets and the l clause gadgets are pairwise vertex-disjoint. To cover a variable edge we need at least 1 of its vertices, and to cover the clause gadget edges we need at least 2 of its vertices. So S has at least $n + 2l$ vertices. Since $|S| = n + 2l$, we require: S contains **exactly one** vertex from each variable gadget, and **exactly two** vertices from each clause gadget. Create an assignment as follows: set $x_i = 1$ if the variable gadget vertex x_i is in S, and $x_i = 0$ if the variable gadget vertex $\overline{x_i}$ is in S.

- Firstly, this is a valid assignment, since exactly one of $x_i, \overline{x_i}$ is in S, so there are no inconsistencies.
- Next, this is a satisfying assignment for ϕ. Consider a clause C_j. Exactly one vertex v of its triangle is not in S. The connection edge from v to its variable-gadget vertex w must be covered, and $v \notin$

S, so $w \in S$. Thus the literal of v is true under our assignment, and so C_j is satisfied. This holds for every clause, so ϕ is satisfied.

This completes the reduction. ■

5. Show that 3SAT many-to-one reduces to Exact-3SAT.

Say 3SAT instances can have at most 3 distinct literals per clause, and EXACT-3SAT is a satisfiability problem where the instance ϕ is in CNF form and it must have **exactly** 3 distinct literals per clause.

The reduction can be shown simply by using the transformations given on page 2–3 to convert any clause with at most 3 literals, to clauses with exactly 3 literals, by adding new variables.

6. Show that Directed Hamiltonian Cycle many-to-one reduces to Directed Hamiltonian Path.

Given a directed graph G, we need to construct a directed graph G', such that

$$G \text{ has a Hamiltonian Cycle if and only if } G' \text{ has a Hamiltonian Path.}$$

To construct G',
- In G, pick any vertex v.
- Remove the vertex v. Add two new vertices v_{in} and v_{out} in G'. If there was a directed edge from u to v in G, add an edge $u \to v_{\text{in}}$ in G'. If there was a directed edge from v to w in G, add an edge $v_{\text{out}} \to w$ in G'.

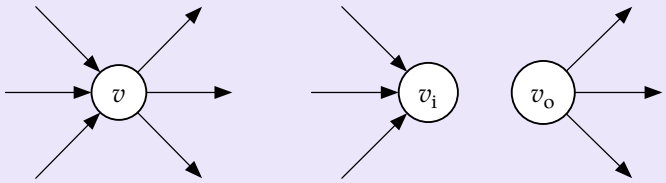


Figure 8: Construction of G' for the reduction DIRHAMCYCLE $\leq_p^m$ DIRHAMPATH.

[$\Rightarrow$] Let the Hamiltonian Cycle in G be $C = \langle v, v_1, v_2, ..., v_k, v\rangle$ (we can arbitrarily start the cycle at any point, say we do it at v). Then, $P = \langle v_{\text{out}}, v_1, v_2, ..., v_k, v_{\text{in}}\rangle$ is a Hamiltonian Path in G'. It clearly passes through every vertex in G', and all edges it uses are valid edges in G'.

[$\Leftarrow$] Let P be a Hamiltonian path in G'. Now, v_{out} does not have any incoming edge in G', but it must be in the path P, so it must be the starting vertex of the path. Similarly, v_{in} does not have any outgoing edge but it should be in P, so it must be the last vertex. So, $P = \langle v_{\text{out}}, v_1, v_2, ..., v_k, v_{\text{in}}\rangle$. Then $C = \langle v, v_1, v_2, ..., v_k, v\rangle$ is a Hamiltonian Cycle in G. It passes through all vertices in G, and all edges are valid edges in G. ■

7. Show that Vertex Cover many-to-one reduces to Subset Sum.

VERTEX-COVER $\leq_p^m$ SUBSET-SUM

Given a graph $G = (V, E)$ and an integer k, we need to construct a list of numbers and a target t, such that

$$G \text{ has a vertex cover of size } k \text{ if and only if some subset of the numbers sums to } t.$$

Let $V = \{v_1, ..., v_n\}$ and $E = \{e_1, ..., e_m\}$ (we assume G has no self-loops and $k \leq n$). We need to create a list of numbers, such that a subset of the numbers will encode a set of vertices, and the digits should tell us whether it has the right size and covers every edge.

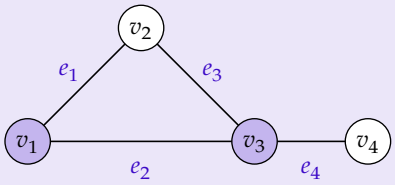
Say we write the numbers in base 10, with $m+1$ digits. The leading digit is a **dummy** column D, which counts the number of chosen vertices, and the remaining m digits are labelled by the edges $e_1, \ldots, e_m$. We create $n+m$ numbers:

- For every vertex v_i, a number a_i with digit 1 in the dummy column, and digit 1 in the column of every edge e_j incident to v_i. All other digits are 0. (For each edge column, among the vertex numbers, the only two numbers which have a digit 1 will correspond to the two endpoints of that edge.)
- For every edge e_j, a *slack* number b_j with digit 0 in the dummy column, and digit 1 in the column of e_j only. All other digits are 0.

The target t has digit k in the dummy column, and digit 2 in each of the m edge columns. (Note that k may not be a valid digit in base 10, essentially the number is $t = k \times 10^m + 2\sum_{j=0}^{m-1} 10^j$; symbolically we represent it as a digit k in the dummy column). It creates $n+m$ numbers with $m+1$ digits each, so it takes polynomial time.

	D	e_1	e_2	...	e_m
a_1	1	1	1		
a_2	1	1			
$\vdots$	$\vdots$				
a_n	1		1		
b_1	0	1	0	...	0
b_2	0	0	1	...	0
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
b_m	0	0	0	...	1
t	k	2	2	...	2

Figure 9: Constructing the list of numbers for the reduction VERTEX-COVER $\leq_p^m$ SUBSET-SUM.



			D	e_1	e_2	e_3	e_4
a_1	=	1	1	1			
a_2	=	1	1			1	
a_3	=	1		1	1	1	
a_4	=	1					1
b_1	=		1				
b_2	=			1			
b_3	=					1	
b_4	=						1
t	=	2	2	2	2	2	

Figure 10: A graph G with its vertex cover, and the corresponding instance of Subset-Sum. (Zeroes not shown for clarity.)

Now we need to show that G has a vertex cover of size k if and only if some subset of these numbers adds up to t.

[$\Rightarrow$] Suppose G has a vertex cover C with $|C| = k$. Choose the numbers a_i for all $v_i \in C$. The dummy column has sum k, as required. Consider the column of an edge $e_j = (u, v)$. Since C is a vertex cover, at least one of u, v is in C, so the chosen vertex numbers contribute 1 or 2 to this column. If the sum is

2, we are done. If it is 1 (exactly one endpoint is in C), also choose the slack number b_j, which brings the column to exactly 2. Therefore, our chosen set of numbers sums to the target t in every column.

[⇐] Suppose there exists a subset of numbers which add up to t. First, notice that carrying can only matter in the dummy column. In each edge column the total sum of digits never exceeds 3. The dummy column is the leading digit, so a carry out of it does not affect the others. So the dummy column adds up to k, and every edge column adds up to 2 independently.

Let S be the set of vertices whose numbers were chosen. The dummy column says exactly that $|S| = k$. We claim S is a vertex cover. Consider an edge e_j. Its column sums to 2, and the slack number b_j is the only non-vertex number that can contribute to it, adding at most 1. So the chosen vertex numbers contribute at least 1 to this column, which means that at least one endpoint of e_j is in S. Since this holds for every edge, S is a vertex cover of size k.

This completes the reduction. ■

8. Show that Subset Sum many-to-one reduces to Partition.

PARTITION: Given a list of numbers $a_1, a_2, ..., a_n$, determine if there exists an equal partition: $S \subseteq \{1, 2, ..., n\}$ such that $\sum_{i \in S} a_i = \sum_{\substack{i \notin S \\ i \in [n]}} a_i$.

SUBSET-SUM $\leq_p^m$ PARTITION

Given a list of numbers $a_1, a_2, ..., a_n$ and a target t, we need to construct another list of numbers, such that

There exists a subset of $\{a_i\}$ that sums to t, if and only if there exists an equipartition of the new list.

Let $A = \sum_i a_i$. Assume $t \leq A$.

To the list of numbers, just add two new numbers, namely $2A - t$ and $A + t$. The sum of the new list of numbers is $4A$.

[⇒] We have $S \subseteq [n]$ such that $\sum_{i \in S} a_i = t$. Then, in the new list consider the subset S along with the new number $2A - t$. Sum of all these numbers is $t + 2A - t = 2A$, thus this is an equipartition of the new list of numbers.

[⇐] First note that the two newly added numbers can't be in the same side of the partition since $2A - t + A + t = 3A > 2A$. Then they must be in different partitions. Consider the subset which includes $2A - t$. Since the entire subset adds up to $2A$, all other numbers in this subset (other than $2A - t$) must sum to t. Thus we have found our required subset.

9. Show that Vertex Cover many-to-one reduces to Hitting Set.

HITTING-SET: Given sets $S_1, S_2, ..., S_m$ and a universe U ($S_i \subseteq U$), a subset $T \subseteq U$, such that $T \cap S_i \neq \emptyset$ for all i, is called a **hitting set**. Does there exist a hitting set of size k?

Given a graph $G = (V, E)$ and a parameter k, we need to create sets $S_1, S_2, ..., S_m$, U and a parameter k' such that

G has a vertex cover of size k if and only if our instance has a hitting set of size k'.

Take $k' = k$. Let $U = V$, and $m = |E|$. For each edge $e_i = (u_i, v_i)$, make the set $S_i = \{u_i, v_i\}$.

[⇒] Let C be a vertex cover of G with $|C| = k$. Then C is also a hitting set of size k in our instance. This is because it 'hits' every set, as each set represents an edge, and each edge is covered by C, so at least one endpoint of the edge must be in C. (At least one element in S_i must be in C.)

[⇐] Let H be a hitting set. Then, for each $S_i = \{u_i, v_i\}$, one of u_i or v_i must be in H. Therefore, for each edge e_i, one of its endpoints is in H. Thus H is a vertex cover.

10. Show that CNF-SAT many-to-one reduces to 3SAT.

Same as 3.

11. Show that 3 Dimensional Matching many-to-one reduces to Set Cover.

3D-MATCHING: Given disjoint sets X, Y and Z, each of size n, and given a set $T \subseteq X \times Y \times Z$ of ordered triples, does there exist a set of n triples in T so that each element of $X \cup Y \cup Z$ is contained in exactly one of these triples?

SET-COVER: Given a set U of n elements, a collection $S_1, \ldots, S_m$ of subsets of U, and a number k, does there exist a collection of at most k of these sets whose union is equal to all of U?

Given an instance of 3D-matching, construct an instance of Set-Cover such that

There exists a perfect 3D-matching, if and only if there exists a set cover of size at most k.

Let $U = X \cup Y \cup Z$, and for each triple $(u, v, w) \in T$, define a set $S_i = \{u, v, w\}$. Let $k = n$.

[⇒] Let M be a perfect 3D matching, then pick the sets corresponding to each triple in M. This collection of sets covers every element in U (by definition of 3D matching).

[⇐] Let C be a set cover of size at most n. Since it covers $3n$ elements, its size must be exactly n. Define M as the set of triples corresponding to each set in C. Since C covers U, every element of $X \cup Y \cup Z$ is contained in at least one triple. Also each triple must be disjoint since otherwise $3n$ elements can't be covered by n triples. So M is a perfect 3D matching.

12. Show that Clique many-to-one reduces to Vertex Cover.

Given in notes.

13. Prove that the Traveling Salesman problem is NP-complete.

TSP: Given a set of distances on n cities, and a bound k, is there a tour of length at most k?

TSP is in NP, since we can use a tour itself as the witness. The verifier checks if the total cost of the tour is at most k and the tour visits every city.

HAMCYCLE $\leq_p^m$ TSP: See CLRS Theorem 34.14 (p1096) or KT Theorem 8.18 (p479).

14. Show that 3SAT many-to-one reduces to 3-Colorability of graphs.

See KT Theorem 8.22 (p487), or CLRS Problem 34-3 (p1103).

15. Show that Subset Sum many-to-one reduces to Knapsack.

(Rough sketch) We can create a knapsack instance by taking same weight and value of each item, $w_i = v_i = a_i$, capacity $W = t$, and target value $V = t$. A subset of items with total value $\geq V$ and total weight $\leq W$ is just a subset with sum t.

16. Show that Subset Sum many-to-one reduces to Partition.

Same as 8.

17. Show that Hamiltonian Cycle many-to-one reduces to Hamiltonian Path.

Here we consider the undirected versions of the problems.

(Rough sketch) We make the following changes in the graph G. Choose any vertex v, duplicate it to v'. Add the same edges as v to v'. Add two new vertices s and t, add edges (s, v) and (v', t). Cutting a Hamiltonian cycle of G at v and adding s and t at the ends gives us a Hamiltonian Path in G'. Conversely, a Hamiltonian path in G' must start at s and end at t, so it goes from v to v' which can be closed into a Hamiltonian cycle in G.

18. Show that Hamiltonian Path many-to-one reduces to Hamiltonian Cycle.

(Rough sketch) Simply add a new vertex p and add edges from p to all other vertices in G. A Hamiltonian path $\langle v_1, ..., v_k \rangle$ can be extended with p to get a Hamiltonian cycle in G'. Similarly, removing p from a Hamiltonian cycle in G' gives a Hamiltonian path in G.

19. Show that Subgraph Isomorphism is NP-complete.

SUBGRAPH-ISO: Given two graphs G and H, does there exist a subgraph S of G such that S and H are isomorphic?

This is in NP by taking the subgraph and the vertex bijection as a witness.

This problem is just a generalisation of CLIQUE, in particular, CLIQUE is a special case of SUBGRAPH-ISO, where H is a complete graph of size k.

CLIQUE $\leq_p^m$ SUBGRAPH-ISO

Given a graph G and a parameter k, we need to give graphs G' and H such that

$$G \text{ has a clique of size } k \text{ if and only if some subgraph of } G' \text{ is isomorphic to } H.$$

Simply take $G' = G$ and $H = K_k$ where K_k denotes the complete graph with k vertices. It is obvious that G has a clique of size k if and only if some k-sized subgraph of G is a complete graph (by definition of a clique).

20. Show that dominating set is NP-complete.

DOMINATING-SET: Does there exist a dominating set of size k in a graph G?

In a graph $G = (V, E)$, a set $S \subseteq V$ is called a **dominating set** if every vertex $v \in V$ is either in S or adjacent to some vertex in S.

VERTEX-COVER $\leq_p^m$ DOMINATING-SET

Given G and k, construct G' and k' such that

$$G \text{ has a vertex cover of size } k \text{ if and only if } G' \text{ has a dominating set of size } k'.$$

Take $k' = k$.

(WLOG, assume the graph G doesn't have any isolated vertices.)

To construct G', for each edge (u, v) in G, add a vertex w_{uv}, and add edges (u, w_{uv}) and (v, w_{uv}). Keep all other vertices and edges intact.

[⇒] Let S be a vertex cover of G. Then S is a dominating set of G'. This is because for every original edge (u, v), one of u or v must have been chosen in S. This vertex covers all three vertices u, v and w_{uv}. Since there are no isolated vertices, we have covered all vertices so this is a dominating set.

[⇐] Let S be a dominating set of G'. First, WLOG we can assume S does not contain any newly added vertices. If it does contain some w_{uv}, we can replace it with u or v, or if both u and v are already in S, just remove w_{uv}. By doing so, S still remains a dominating set.

Then, we claim that S is a vertex cover. Suppose it doesn't cover some edge (u, v), that means both u and v are not in this dominating set. Then, w_{uv} is not in the dominating set, nor adjacent to any vertex in it, which is a contradiction.

21. Prove the following problems to be NP-complete: Hamiltonian Cycle, 3-Dimensional Matching, 3-Colorability of graphs.

Hamiltonian Cycle: 3SAT $\leq_p^m$ HAMCYCLE: KT book, Theorem 8.17 (p475)

3D Matching: 3SAT $\leq_p^m$ 3D-MATCHING: KT book, Theorem 8.20 (p482)

3-Colorability: Same as 14.